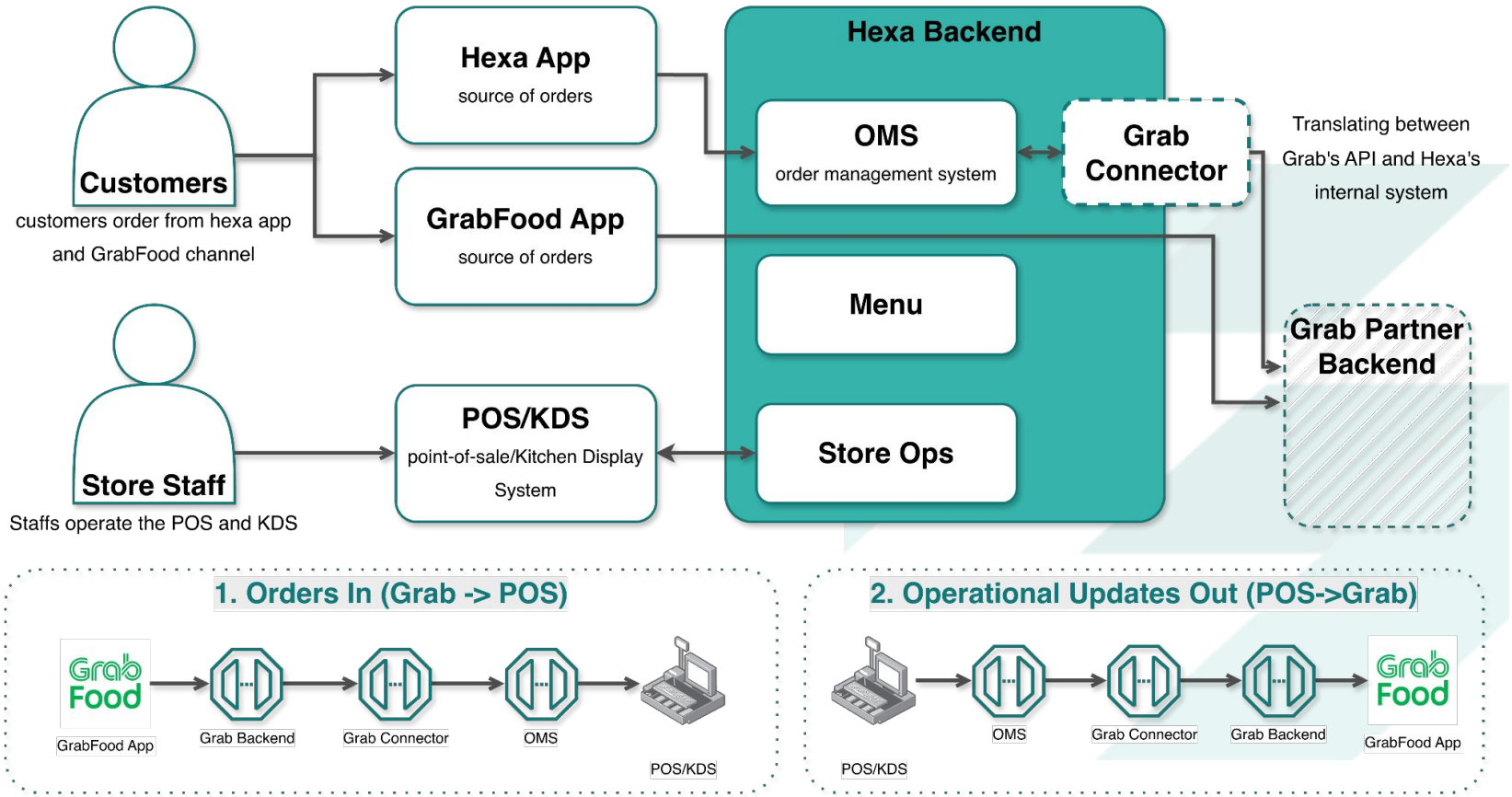
REST in the Real World: Designing from scratch

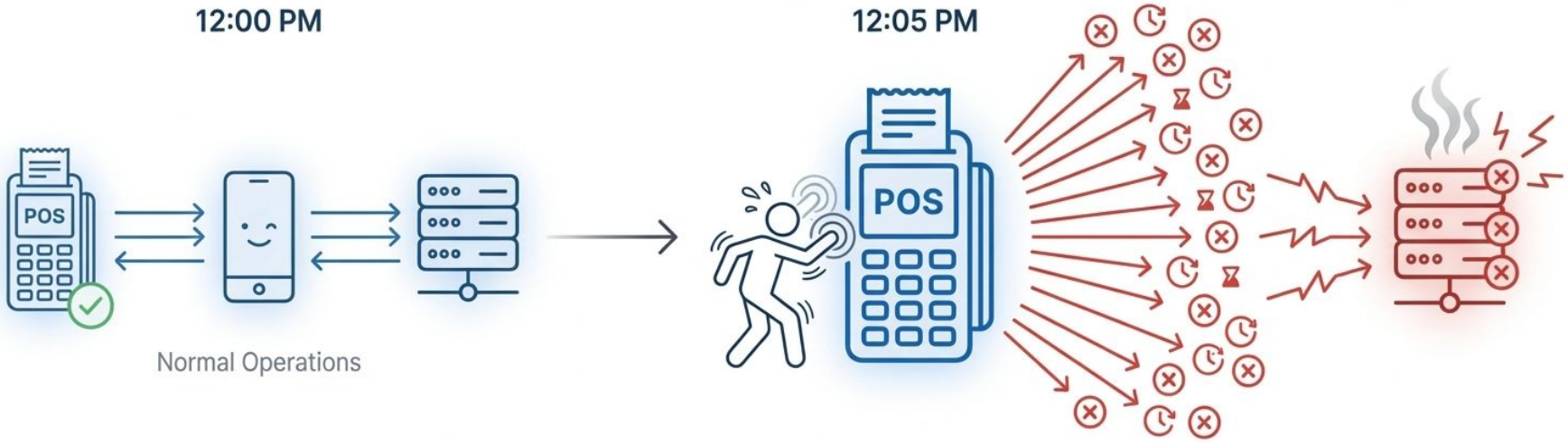
Outcome:

- Understanding of common integration issues in distributed systems
- A **workflow** to systematically translate business problem into API resources.
- A **decision tree** to choose the right endpoint shapes and metrics.

Example System Context



The Rush Hour



Symptom

POS terminals time out when staff tap the “Mark Ready” button.

Human Reaction

Staff, under pressure, taps the button again. And again. This floods the system with duplicate requests.

Systemic Failure

Some orders get stuck in a ‘Preparing’ state on the customer’s app, even though the food is ready. Others seem to vanish entirely during pagination on the KDS.

Business Impact

Support tickets explode. Angry customers, frustrated staff, and lost revenue.

Example of Integration error

Kinh doanh

06/01/2026 22:41 GMT+7

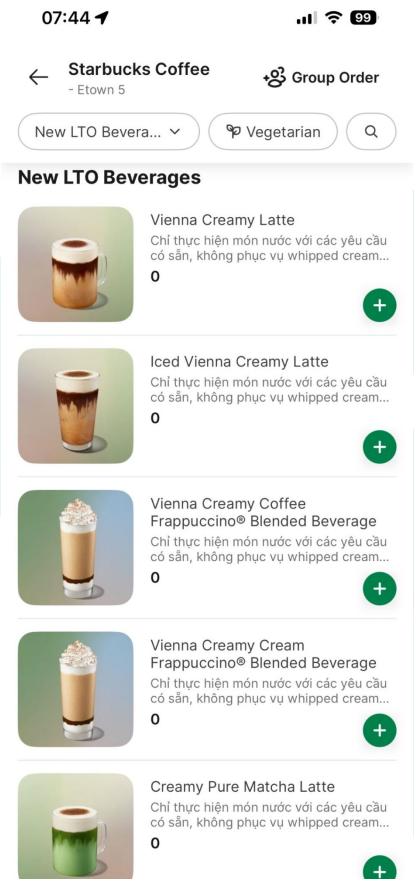
Starbucks xin lỗi vì sự cố giá 0 đồng, khách hàng thông cảm

NGHI VŨ

Nghe đọc bài 3:32 1.0 Nữ miền Nam

Tuổi Trẻ trên Google News

Trong sự cố này, giá nước của Starbucks được đặt với giá 0 đồng trên ứng dụng giao hàng, nhiều khách hàng đã đặt hàng thành công, nhưng nhiều shipper lên tiếng khi họ phải đợi lâu, bị hủy đơn.



Example of Integration error

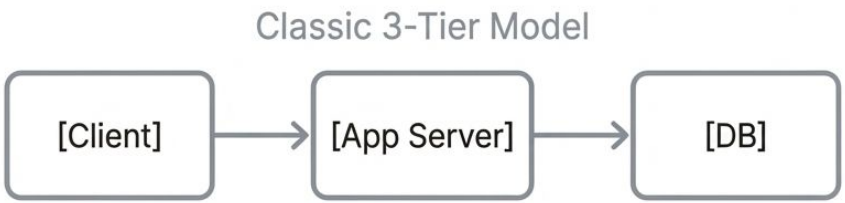
GrabDeveloper Products **Docs** Contact Us

Items have the following rules:

- **A category should have minimum 1, maximum 300 items**: If there is less than 1 item, you will get an error. If there are more than 300 items, GrabFood truncates the rest.
- Item `name` has a character limit of **80**.
- Item `description` has a default character limit of **300** for all countries, except for `VN`, which has a custom limit of **2000** characters. Any characters exceeding the limit will be truncated.
- Item `price` is mandatory. If not provided, the price will be **set to 0** by default.
  - For clarification on whether the menu price is inclusive or exclusive of tax, please refer to the FAQs.
- Item `availableStatus` defines the visibility on the Grab app platform and correlated with `maxStock`.
  - For clarification on how each availableStatus is reflected and how maxStock impacts, please refer to the FAQs.
- Only **1** `photos` is supported per item.
  - For recommended image formats, please refer to the FAQs.

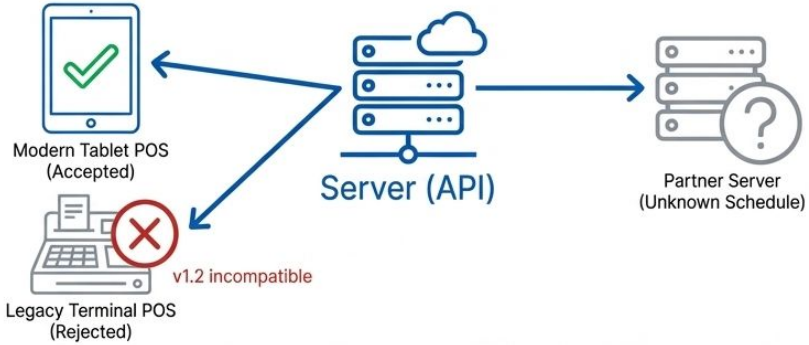
Coupling and the Myth of Instant Updates

The Assumption



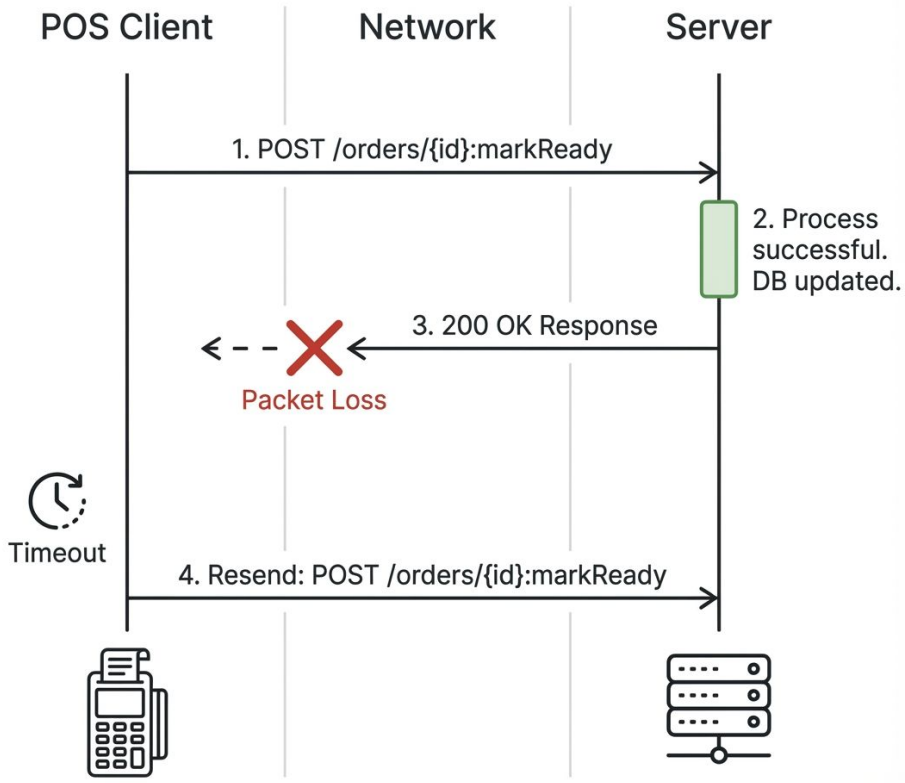
- Scale by adding app servers
- DB is the shared truth
- One team, one release cycle
- Works great for single-channel web apps

The Reality



- POS versions roll out slowly across stores
- Partners / vendors implement differently
- If endpoint behavior changes, you can't "force update" everyone

When a Timeout Isn’t a Failure

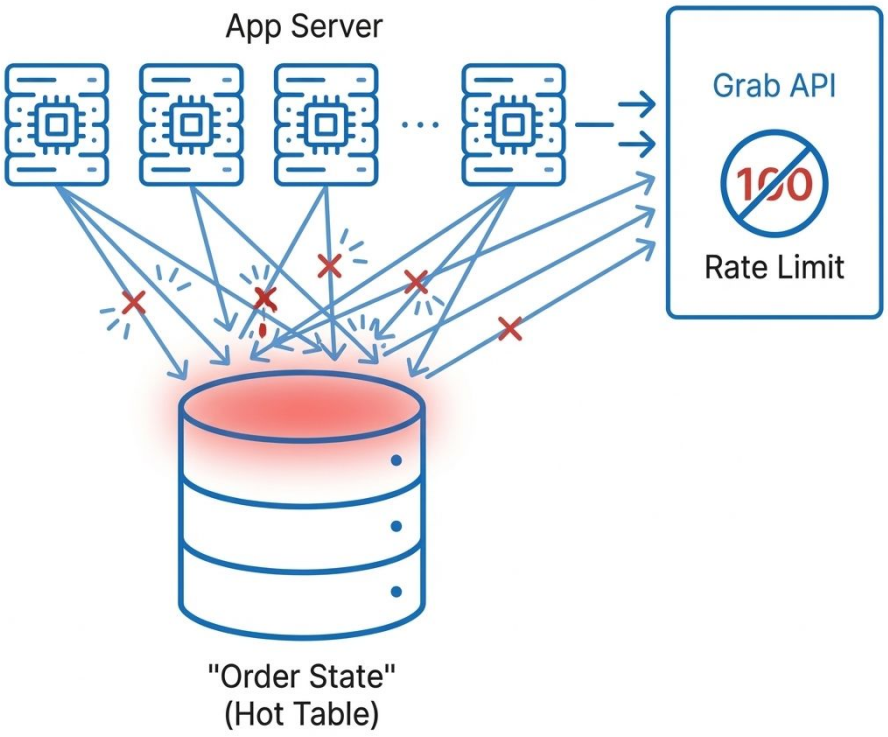


The classic 3-Tier assumption:
If the client gets an error or timeout, the operation probably failed on the server.

The Integration reality:

- A timeout means the state is **unknown.** The server might have succeeded. Retries are inevitable and often automatic (clients, GWs, webhooks)
- Without safe retry mechanism, API creates business errors. Duplicate POST might trigger two notifications or write duplicate audit logs
- **Idempotency** becomes the core requirement for business correctness

The Scaling bottlenecks aren’t where you think



The classic 3-Tier assumption:
To handle more load, just add more stateless application servers behind a LB

The Integration reality:

- **DB Contention**: Thousands of concurrent POS terminals trying to update the status of orders.
- **Weakest link**: The system is only as fast as the partner API we are calling. If Grab API slows down, the entire system slows down.
- **Retry storm:** When a downstream service is failing, upstream callers initiates a massive and synchronized retry storm.

REST architecture style

REST is a set of architectural constraints that gives us powerful levers for building systems that survives real-world conditions.

Resources & Representations (`/orders/{id}`)

Modeling your domain with nouns, not verbs, creates stable contracts that are decoupled from specific business workflows. This directly addresses the **coupling** problem.

Stateless Requests

Each request contains all information needed to process it. This allows for robust scaling, failover, and simplified caching.

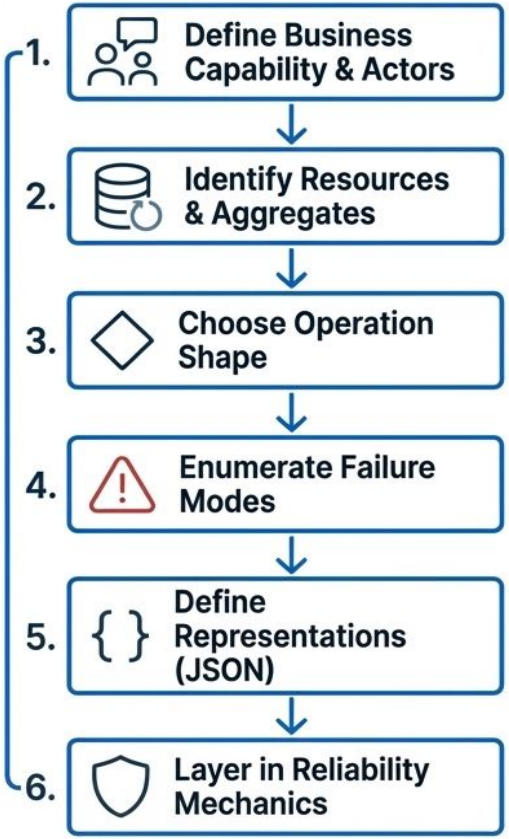
Uniform Interface (HTTP Verbs)

Consistent use of `GET`, `POST`, `PUT`, `PATCH` provides predictable semantics for clients, gateways, and developers, solving the **intermediary semantics** problem.

Standard Semantics (Idempotency, Safety)

The properties of HTTP verbs provide a built-in grammar for handling **retries** and network failures correctly.

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

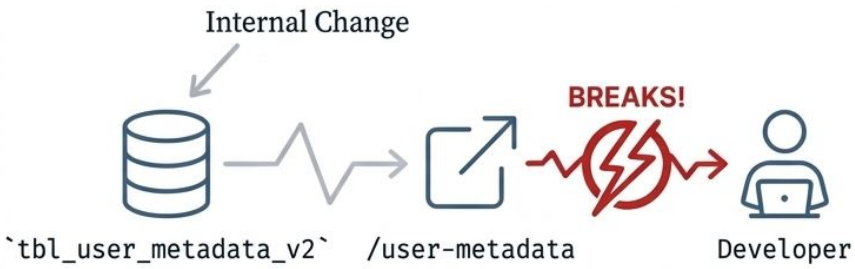
5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

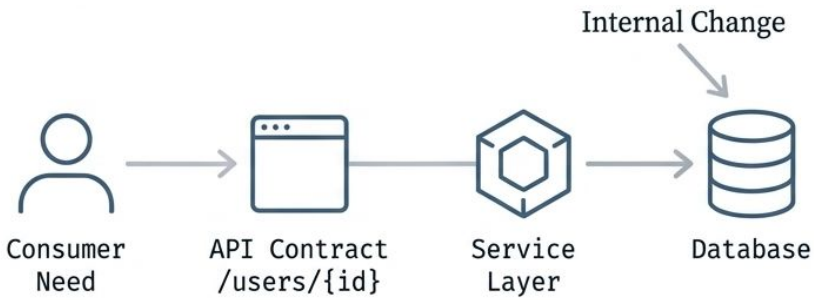
Step 1: Modeling the business capability

Principle I: Prefer API-first development

The Leaky Abstraction (Inside-Out)



The Resilient Contract (Outside-In)



Step 1: Modeling the business capability

Example Business capability: Order lifecycle updates from POS

Ubiquitous Language

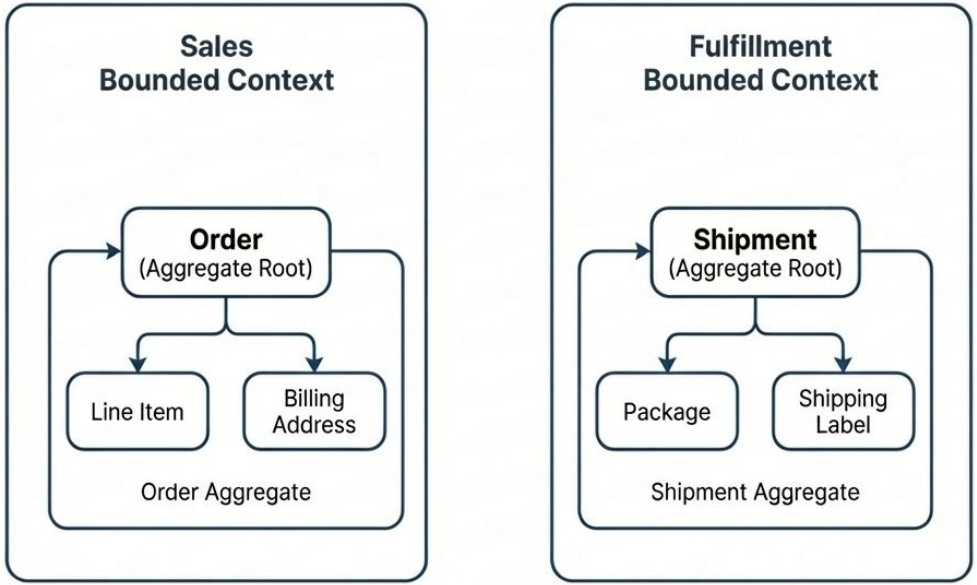
Use the language of the business, not technical jargon. If domain experts call it a 'Shipping Order', your resource is `/shipping-orders`, not `/shipment_tbl_v3`.

Aggregate

A cluster of objects treated as a single unit for data changes. An external client should only interact with the 'Aggregate Root'.

Bounded Context

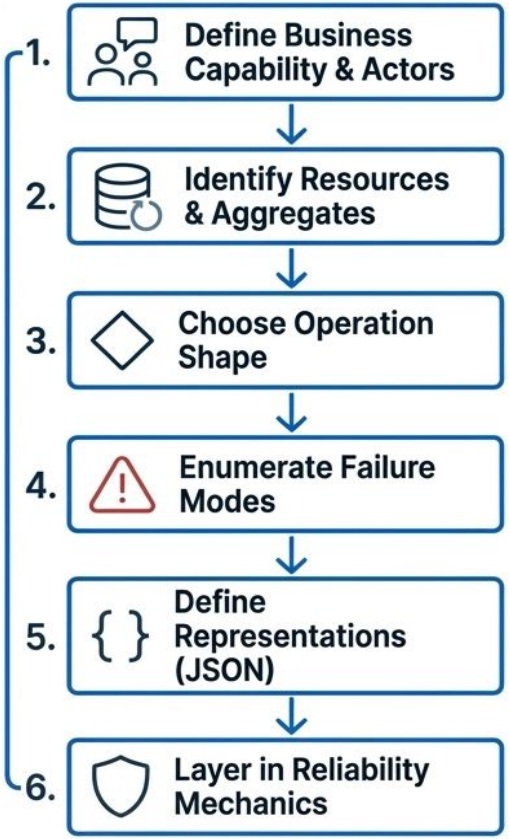
Explicitly defines the boundaries of your model. An 'Order' in the 'Sales' context is different from an 'Order' in the 'Fulfillment' context. This prevents 'god-objects' and clarifies microservice responsibilities.



Step 1: Modeling the business capability

Example Business capability: Order lifecycle updates from POS. This must be retry-safe and respect the Order State Machine (CANCELLED cannot transition to READY).

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

Step 2: Identify resources / aggregates

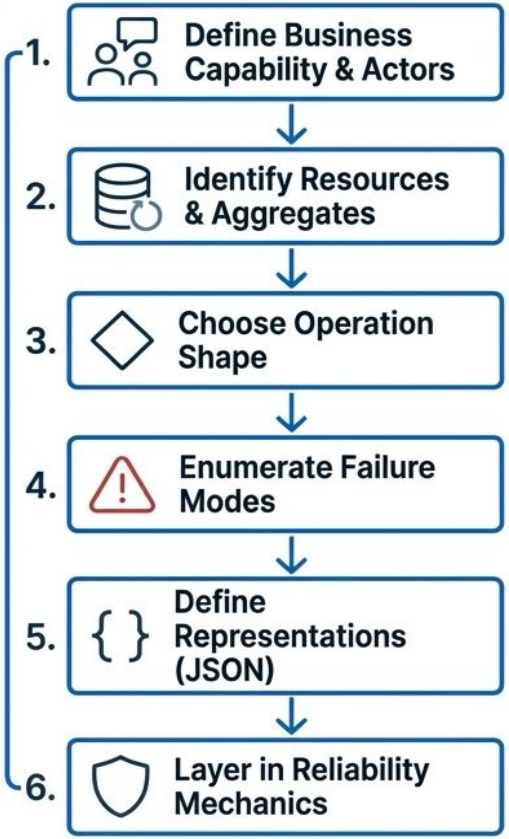
Example Business capability: Order lifecycle updates from POS. This must be retry-safe and respect the Order State Machine (CANCELLED cannot transition to READY).

=> Resources: Order, Store, Menu Sync Job, Staff, …

=> Key operations:

- accept/reject
- update ready time
- **mark ready**
- cancel (with cancelable check)
- list orders

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

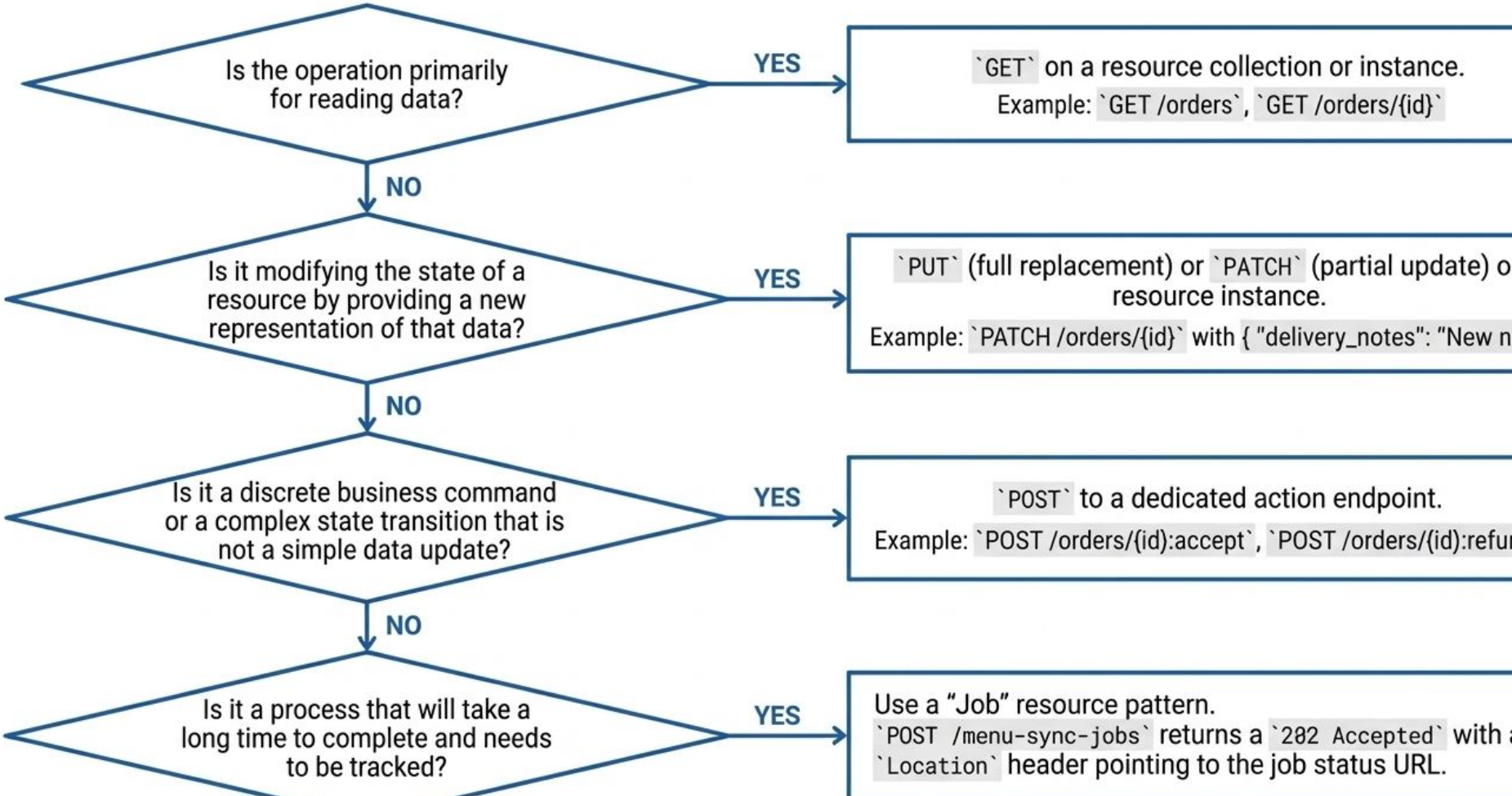
2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

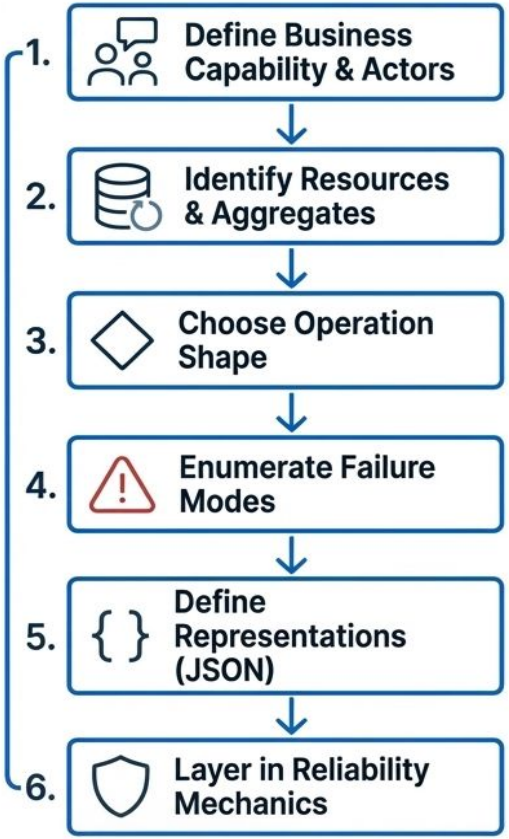

Is the operation primarily for reading data?
YES
`GET` on a resource collection or instance.
Example: `GET /orders`, `GET /orders/{id}`
NO
Is it modifying the state of a resource by providing a new representation of that data?
YES
`PUT` (full replacement) or `PATCH` (partial update) o
resource instance.
Example: `PATCH /orders/{id}` with { "delivery_notes": "New n
NO
Is it a discrete business command or a complex state transition that is not a simple data update?
YES
`POST` to a dedicated action endpoint.
Example: `POST /orders/{id}:accept`, `POST /orders/{id}:refu
NO
Is it a process that will take a long time to complete and needs to be tracked?
YES
Use a "Job" resource pattern.
`POST /menu-sync-jobs` returns a `202 Accepted` with a
`Location` header pointing to the job status URL.

#3: Decision tree #1: Resource vs Action vs Job

RPC-ish Style (Verbs)	Resource-Update Style (Nouns)	Action Endpoint Style (Hybrid)
POST /markOrderReady `{"order_id": "123"}`	**PATCH /orders/{id}** `{"status": "READY"}`	**POST /orders/{id}:accept** *(No body needed for simple action)*
Pros: ✓ Very explicit and simple to call. **Cons:** ✗ Endpoint explosion (`/acceptOrder`, `/cancelOrder`, etc.). ✗ Unclear idempotency rules by default. ✗ Tightly couples clients to the workflow.	**Pros:** ✓ Pure RESTful model. ✓ Single endpoint for all status changes. **Cons:** ✗ Can expose state machine logic to clients. ✗ Risk of clients sending invalid states. ✗ `PATCH` semantics can be complex.	**Pros:** ✓ Explicit command captures business intent. ✓ Encapsulates complex server-side rules. ✓ Safer for partners than a free-form `PATCH`. **Cons:** ✗ Not purely "RESTful" (but pragmatic). ✗ Can still lead to many endpoints if overused.

Conclusion: For partner integrations, the **Action Endpoint** style often wins. It combines the explicitness of RPC with the resource-centric model of REST, providing the best balance of clarity and safety.

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

Vague & Unhelpful

```
HTTP 500
{
  "error": "Something went wrong"
}
```

Structured & Actionable (RFC 9457

```
HTTP 400
{
  "type": "https://api.example.com/errors/valid
  "title": "Validation Failed",
  "status": 400,
  "detail": "Request body contains invalid fiel
  "instance": "/habits",
  "errors": [...]
}
```

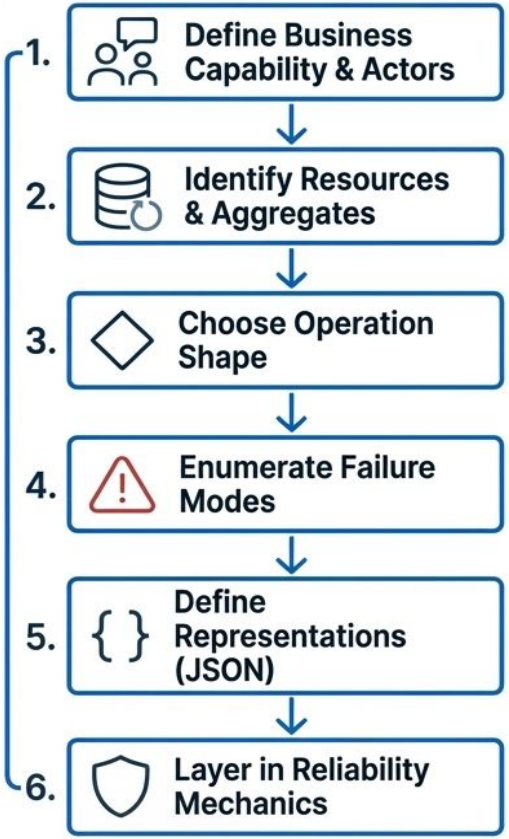
The Standard: **MUST** Use Problem Details (RFC 9457). It provides a standard, machine-readable forma
to understand what broke, why it broke, and how to fix it.

Essential Status Codes Quick Reference

400 Bad Request: Client sent invalid data.
401 Unauthorized: Authentication required.
403 Forbidden: Authenticated, but not authorized.
404 Not Found: Resource does not exist.

429 Too Many Requests: Rate limit exceeded
500 Internal Server Error: We screwed up (n
stack traces).

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

#5 Define request/response + error model

MUST

Always return a JSON object as the top-level structure, never a raw array. This allows for future additions like pagination metadata without breaking clients.
(e.g., {\"data\": [...]} not [...]])

MUST

Dates and times must conform to RFC 3339 / ISO 8601 format (2026-01-15T10:00:00Z).

```
{"data":
  [{
    {"id": "e_8YH",
    "habitId": "code-review",
    "createdAt": "2025-08-08T09:17:34Z"
  }],
  "pagination": {
    "nextCursor": "cursor_01J9KaBcd",
    "hasMore": true
  }
}
```

#5 Define request/response + error model

Response samples

200 400 403 404 409

500

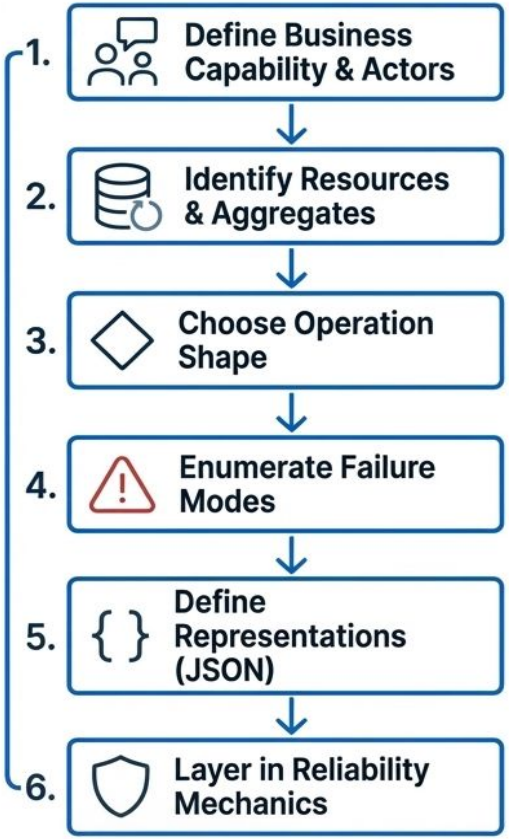
Content type

application/json

Copy

```
{
    "target": "",
    "reason": "error_reason",
    "message": "error detail"
}
```

REST API Design Workflow



1. **Define Business Capability & Actors**
   What goal is the user trying to achieve? (e.g., "Store staff need to inform the customer that an order is ready for pickup.")

2. **Identify Resources & Aggregates**
   What are the core nouns (domain concepts) involved? An `Order` is a clear Aggregate Root. A `Store` is another. The goal is to find the "Consistency Boundary." (Source: IBM DDD)

3. **Choose Operation Shape**
   Is this a simple data read (`GET`), a direct state update (`PATCH`), a complex business command (`POST` to an action endpoint), or a long-running task (Job resource)?

4. **Enumerate Failure Modes**
   What happens if the request is invalid? The user is unauthorized? A business rule is violated (e.g., trying to cancel an order that's already delivered)?

5. **Define Representations (JSON)**
   Design the request/response bodies. Use consistent naming (`snake_case`), standard formats (ISO 8601 dates), and always return JSON objects as top-level structures to support future extensibility. (Source: HMCTS)

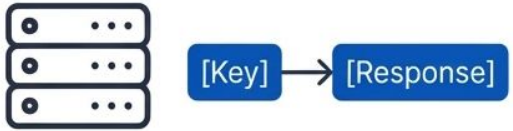
6. **Layer in Reliability Mechanics**
   Add idempotency, concurrency control (ETags), and pagination.

#6 Mechanics tree: Hardening Write ops

For Retries (Idempotency)

Can this request be retried by the client or network? (For a POS integration, the answer is always **YES**).

Require an `Idempotency-Key: <UUID>` header on all `POST`, `PUT`, and `PATCH` requests.



The server stores the key and the first response for a limited time (e.g., 24 hours).

- Subsequent requests with the same key receive the cached response without re-processing the operation.

For Conflicts (Concurrency Control)

Can two actors (e.g., store staff at POS, an automated system) try to update the same resource at once? (Again, **YES**).

Use Optimistic Locking with ETags.

1. GET → ETag Header → 2. PATCH + If-Match Header → 3. Server Compares ETags

A GET response for a resource includes an `ETag: "v2-hash"` header.

- Any PATCH or PUT request to that resource **must** include an `If-Match: "v2-hash"` header.
- If the ETag on the server has changed, the request is rejected with `412 Precondition Failed`, forcing the client to re-fetch the resource.

#6 Mechanics tree: listing & pagination

If your primary need is...	Then use...	Why?
Stable, reliable paging through high-churn data (like a live order list).	**Cursor-based Pagination** `GET /orders?limit=20&cursor=...`	The cursor is a stable pointer to a specific item in the dataset. New orders being added before or after the cursor do not affect the next page's results. Essential for correctness.
Allowing a user to jump to a specific page number (rare for operational UIs, more common for analytics).	**Offset-based Pagination** `GET /orders?limit=20&offset=40`	Simpler to implement, but you **must** document the risk of skipped/duplicate items if the underlying data changes during pagination. Unsafe for high-churn scenarios.

PR Checklist

1. **Resource Model:** Are resources modeled as **nouns** from the **business domain**? Is the endpoint shape (Resource vs. Action) chosen **intentionally**?
2. **Idempotency:** Are all write operations (`POST`, `PUT`, `PATCH`) **safe to retry** via an **`Idempotency-Key`**?
3. **Concurrency:** Is there a strategy to **prevent lost updates** when **multiple actors are involved** (e.g., `ETag` + `If-Match`)?
4. **Errors:** Are error responses **structured** (e.g., Problem+JSON), using standard HTTP codes, and providing **actionable details**?
5. **Pagination:** For collections, is the pagination strategy (Cursor vs. Offset) appropriate for the data's expected churn rate?
6. **Evolvability:** Is the API designed for **additive changes** to avoid breaking versions? Are top-level rel responses always JSON objects?

Appendix: REST vs gRPC vs GraphQL

Use Case	Recommended Protocol	Why?
Public or Partner-Facing APIs (Like our POS Integration)	**REST**	**Universal & Standard.** It's built on HTTP, easy for anyone to explore with simple tools, and has a massive ecosystem. The de-facto standard for external communication.
High-Performance Internal Microservices (e.g., an internal pricing engine calling an inventory service)	**gRPC**	**Maximum Performance.** Extremely low latency and small binary payloads via HTTP/2 and Protocol Buffers. Ideal for high-throughput, latency-sensitive, internal service-to-service calls.
Complex Frontends or Mobile Apps (e.g., a dashboard aggregating data from 5 different services)	**GraphQL**	**Data Fetching Flexibility.** Allows clients to request exactly the data they need in a single network call, reducing round-trips and eliminating the over-fetching common in REST APIs.